

```

# =====
# BLOB Principe - SAUT 3 : Chaleur Blob reformulee
# =====
#
# Objectif scientifique :
#   D est partiellement falsifie comme "chaleur" (D atteint pic puis decroit).
#   Trouver une observable qui se comporte comme une chaleur thermodynamique
#   au sens : croit monotone avec la matiere (n_tetra), sature en coherence
#   avec r3 -> 4.65, additive, reproductible.
#
# Trois candidats testes :
#   PHI - Friction tetraedrique = paires de tetras partageant arete / n_tetra
#   PSI - Entropie locale moyenne = moyenne Shannon des degres voisins
#   THETA - Taux de transitions = swaps acceptes par pas / n_tetra
#
# Tests par seed et par observable :
#   T1 - Croissance monotone avec n_tetra
#   T2 - Saturation/ralentissement en grande N
#   T3 - Reproductibilite : CV < 30% entre seeds
#   T4 - Additivite (test approxime via sous-graphes)
#
# Verdict : selection du meilleur candidat satisfaisant les 4 criteres
# =====

import sys
import os

# =====
# AUTO-TEST CORRUPTION
# =====
def auto_check():
    print("=" * 70)
    print("  AUTO-TEST CORRUPTION CARACTERES")
    print("=" * 70)
    try:
        if hasattr(sys, 'argv') and len(sys.argv) > 0 and os.path.exists(sys.argv
            source_file = sys.argv[0]
        else:
            source_file = __file__ if '__file__' in globals() else None
        if source_file and os.path.exists(source_file):
            with open(source_file, 'r', encoding='utf-8') as f:
                content = f.read()
            dangerous = {
                chr(8216): "U+2018", chr(8217): "U+2019",

```

```

        chr(8220): "U+201C", chr(8221): "U+201D",
        chr(8211): "U+2013", chr(8212): "U+2014",
    }
    found = [(name, content.count(c)) for c, name in dangerous.items() if
    if found:
        print("ERREUR caracteres corrompus :", found)
        return False
    star = chr(42)
    bad_init = star + star + "init" + star + star
    if content.count(bad_init) > 1:
        print("ERREUR : motif corrompu init")
        return False
    triple_back = chr(96) * 3
    if content.count(triple_back) > 1:
        print("ERREUR : Markdown")
        return False
    print("OK : pas de corruption")
else:
    print("Note : auto-test partiel")
except Exception as e:
    print("Note : " + str(e))
print("=" * 70)
return True

if not auto_check():
    sys.exit(1)

# =====
# IMPORTS
# =====
import numpy as np
import time
import json
import gc
from collections import Counter

# =====
# Parametres canoniques
# =====
KAPPA_0_TARGET = 8.0
BETA = 0.30
TAU = 0.50
P_MITOSE = 0.001
EPS_DECAL = 0.05

```

```

OUTPUT_DIR = '/content/blob_saut3_results'
os.makedirs(OUTPUT_DIR, exist_ok=True)

# =====
# Classe BlobUniverse avec calcul Phi/Psi/Theta
# =====
class BlobUniverse:

    def __init__(self, seed=0):
        self.rng = np.random.default_rng(seed)
        self.seed = seed
        self.positions = [np.array([0.5, 0.5, 0.5])]
        self.adj = [set()]
        self.edge_list = []
        self.deg = np.array([0], dtype=np.int32)
        self.n_tri = 0
        self.n_tetra = 0
        self.S_curr = 0.0
        self.Q3_dissipation = 0.0
        self.n_mitoses = 0
        self.n_swaps_accepted = 0
        self.first_edge_step = None
        self.first_triangle_step = None
        self.first_tetrahedron_step = None

    @property
    def N(self):
        return len(self.positions)

    @property
    def E(self):
        return len(self.edge_list)

    def _compute_D(self):
        if self.N == 0:
            return 0.0
        mu = float(self.deg.mean())
        if mu == 0:
            return 0.0
        return float(self.deg.std()) / mu

    def _compute_S(self):
        return np.log(1 + self.n_tri) - BETA * self._compute_D()

    def _rips_radius(self):
        if self.N <= 1:

```

```

        return 0.5
    c_3 = 4 * np.pi / 3
    return 1.5 * (KAPPA_0_TARGET / (c_3 * self.N)) ** (1.0/3)

def _count_tetra_through_edge(self, u, v):
    common = self.adj[u] & self.adj[v]
    if len(common) < 2:
        return 0
    common_list = list(common)
    count = 0
    for i, w in enumerate(common_list):
        adj_w = self.adj[w]
        for x in common_list[i+1:]:
            if x in adj_w:
                count += 1
    return count

def _list_tetrahedra(self):
    """Liste exhaustive des tetraedres comme tuples ordonnes (u,v,w,x)."""
    result = []
    seen = set()
    for u, v in self.edge_list:
        common = self.adj[u] & self.adj[v]
        if len(common) < 2:
            continue
        common_list = sorted(common)
        for i, w in enumerate(common_list):
            for x in common_list[i+1:]:
                if x in self.adj[w]:
                    key = tuple(sorted([u, v, w, x]))
                    if key not in seen:
                        seen.add(key)
                        result.append(key)
    return result

# =====
# OBSERVABLES CANDIDATES POUR LA CHALEUR
# =====

def compute_PHI(self):
    """
    PHI = friction tetraedrique
    = nombre moyen de tetraedres partageant une arete avec un tetra donne
    Plus PHI est grand, plus les tetras sont encombres.
    """
    if self.n_tetra == 0:
        return 0.0

```

```

tetras = self._list_tetrahedra()
if not tetras:
    return 0.0

# Pour chaque arete, compter combien de tetras la contiennent
edge_to_tetras = {}
for tet in tetras:
    for i in range(4):
        for j in range(i+1, 4):
            e = (tet[i], tet[j])
            edge_to_tetras.setdefault(e, []).append(tet)

# PHI = nb moyen de "voisins tetraedriques" par tetra
# 2 tetras sont voisins s ils partagent au moins une arete
neighbors_count = {tet: set() for tet in tetras}
for e, tlist in edge_to_tetras.items():
    if len(tlist) > 1:
        for i, t1 in enumerate(tlist):
            for t2 in tlist[i+1:]:
                neighbors_count[t1].add(t2)
                neighbors_count[t2].add(t1)

total_neighbors = sum(len(v) for v in neighbors_count.values())
return float(total_neighbors) / len(tetras)

def compute_PSI(self):
    """
    PSI = entropie locale moyenne
    = moyenne sur tous les sommets v de Shannon(degrees des voisins de v)
    """
    if self.N == 0:
        return 0.0
    total_ent = 0.0
    n_active = 0
    for v in range(self.N):
        neighbors = self.adj[v]
        if not neighbors:
            continue
        n_active += 1
        local_degs = [self.deg[u] for u in neighbors]
        counts = Counter(local_degs)
        total = len(local_degs)
        ent = 0.0
        for c in counts.values():
            p = c / total
            if p > 0:
                ent -= p * np.log(p)

```

```

        total_ent += ent
    return float(total_ent / n_active) if n_active > 0 else 0.0

def compute_THETA(self, n_swaps_in_window, window_size):
    """
    THETA = taux de transitions par tetraedre
    = (nb swaps acceptes dans la fenetre / window_size) / n_tetra
    Mesure l'activite dynamique normalisee par la matiere.
    """
    if self.n_tetra == 0 or window_size == 0:
        return 0.0
    return float(n_swaps_in_window) / window_size / self.n_tetra

# =====
# DYNAMIQUE
# =====

def mitose(self):
    if self.N == 0:
        return
    parent_idx = int(self.rng.integers(self.N))
    parent_pos = self.positions[parent_idx]
    r_p = self._rips_radius()
    decal = self.rng.normal(0, EPS_DECAL * r_p, size=3)
    new_pos = parent_pos + decal
    self.positions.append(new_pos)
    self.adj.append(set())
    self.deg = np.append(self.deg, 0)
    self.n_mitoses += 1

def step(self, current_step):
    if self.rng.random() < P_MITOSE:
        self.mitose()
        return False
    if self.N < 2:
        return False
    if self.E == 0:
        return self._step_add_only(current_step)
    E_target = int(KAPPA_0_TARGET * self.N / 2)
    if self.E < E_target:
        deficit_ratio = (E_target - self.E) / max(E_target, 1)
        if self.rng.random() < deficit_ratio:
            return self._step_add_only(current_step)
    return self._step_swap(current_step)

def _step_add_only(self, current_step):
    u = int(self.rng.integers(self.N))

```

```

v = int(self.rng.integers(self.N))
if u == v or v in self.adj[u]:
    return False
u, v = min(u, v), max(u, v)
dt_add = len(self.adj[u] & self.adj[v])
self.adj[u].add(v)
self.adj[v].add(u)
self.deg[u] += 1
self.deg[v] += 1
new_n_tri = self.n_tri + dt_add
dtetra_add = self._count_tetra_through_edge(u, v)
new_S = np.log(1 + new_n_tri) - BETA * self._compute_D()
delta_S = new_S - self.S_curr
log_alpha = self.N * delta_S / TAU
accepted = (log_alpha >= 0) or (self.rng.random() < np.exp(max(log_alpha,
if accepted:
    self.edge_list.append((u, v))
    self.n_tri = new_n_tri
    self.n_tetra += dtetra_add
    self.S_curr = new_S
    self.n_swaps_accepted += 1
    self.Q3_dissipation += abs(delta_S)
    if self.first_edge_step is None and self.E > 0:
        self.first_edge_step = current_step
    if self.first_triangle_step is None and self.n_tri > 0:
        self.first_triangle_step = current_step
    if self.first_tetrahedron_step is None and self.n_tetra > 0:
        self.first_tetrahedron_step = current_step
    return True
else:
    self.adj[u].discard(v)
    self.adj[v].discard(u)
    self.deg[u] -= 1
    self.deg[v] -= 1
    return False

def _step_swap(self, current_step):
    idx = int(self.rng.integers(self.E))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return False
    u_add, v_add = min(u_add, v_add), max(u_add, v_add)
    if (u_add, v_add) == (u_rm, v_rm):
        return False
    dt_rm = len(self.adj[u_rm] & self.adj[v_rm])

```

```

dtetra_rm = self._count_tetra_through_edge(u_rm, v_rm)
self.adj[u_rm].discard(v_rm)
self.adj[v_rm].discard(u_rm)
self.deg[u_rm] -= 1
self.deg[v_rm] -= 1
dt_add = len(self.adj[u_add] & self.adj[v_add])
self.adj[u_add].add(v_add)
self.adj[v_add].add(u_add)
self.deg[u_add] += 1
self.deg[v_add] += 1
dtetra_add = self._count_tetra_through_edge(u_add, v_add)
new_n_tri = self.n_tri - dt_rm + dt_add
if new_n_tri < 0:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False
new_S = np.log(1 + new_n_tri) - BETA * self._compute_D()
delta_S = new_S - self.S_curr
log_alpha = self.N * delta_S / TAU
accepted = (log_alpha >= 0) or (self.rng.random() < np.exp(max(log_alpha,
if accepted:
    self.edge_list[idx] = (u_add, v_add)
    self.n_tri = new_n_tri
    self.n_tetra += (dtetra_add - dtetra_rm)
    self.S_curr = new_S
    self.n_swaps_accepted += 1
    self.Q3_dissipation += abs(delta_S)
    if self.first_triangle_step is None and self.n_tri > 0:
        self.first_triangle_step = current_step
    if self.first_tetrahedron_step is None and self.n_tetra > 0:
        self.first_tetrahedron_step = current_step
    return True
else:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1

```



```

        return False

# =====
# RUN UNE SEED
# =====
def run_seed_saut3(seed, total_steps):
    t_start = time.time()
    sim = BlobUniverse(seed=seed)

    n_log_samples = 100 # moins de samples car PHI/PSI sont chers
    log_sample_steps = sorted(set([
        max(1, int(np.exp(np.log(total_steps) * i / n_log_samples)))
        for i in range(1, n_log_samples + 1)
    ]))
    next_log = 0

    trajectory = []

    print(' Seed ' + str(seed) + ' init...', flush=True)

    last_n_swaps = 0
    last_step = 0

    for step in range(total_steps):
        sim.step(step + 1)

        if next_log < len(log_sample_steps) and (step + 1) >= log_sample_steps[next_log]:
            # Calcul PHI (cher : ne le faire qu aux echantillons logarithmiques)
            phi = sim.compute_PHI()
            psi = sim.compute_PSI()
            theta = sim.compute_THETA(
                sim.n_swaps_accepted - last_n_swaps,
                (step + 1) - last_step
            )
            last_n_swaps = sim.n_swaps_accepted
            last_step = step + 1

        traj_pt = {
            'step': step + 1,
            'N': sim.N, 'E': sim.E,
            'n_tri': sim.n_tri, 'n_tetra': sim.n_tetra,
            'D': sim._compute_D(),
            'PHI': phi,
            'PSI': psi,
            'THETA': theta,
            'Q3': sim.Q3_dissipation,

```

```

        'time_s': time.time() - t_start,
    }
    trajectory.append(traj_pt)
    next_log += 1

total_time = time.time() - t_start

# Recount final
tri_recount = sum(len(sim.adj[u] & sim.adj[v]) for u, v in sim.edge_list) //
coherent = (sim.n_tri == tri_recount)

# =====
# ANALYSE PAR OBSERVABLE
# =====
# Ne considerer que les points avec n_tetra > 0
valid = [t for t in trajectory if t['n_tetra'] > 0]

def analyze_observable(name, key):
    if not valid:
        return None
    vals = [t[key] for t in valid]
    ntetras = [t['n_tetra'] for t in valid]

    # T1 : monotonie avec n_tetra
    # On regarde si vals augmente quand ntetra augmente
    # Test simple : correlation rang Spearman approximee par correlation simp
    if len(vals) > 2 and np.std(vals) > 0 and np.std(ntetras) > 0:
        corr = float(np.corrcoef(ntetras, vals)[0, 1])
    else:
        corr = 0.0

    # T2 : saturation/ralentissement (concavite)
    # Comparer croissance 1ere moitie vs 2eme moitie
    n = len(valid)
    if n >= 4:
        v_mid = vals[n // 2]
        v_end = vals[-1]
        v_start = vals[0]
        half_growth = v_mid - v_start
        second_growth = v_end - v_mid
        if abs(half_growth) > 1e-9:
            slowdown_ratio = second_growth / half_growth
        else:
            slowdown_ratio = 1.0
    else:
        slowdown_ratio = None

```

```

    return {
        'corr_with_ntetra': corr,
        'monotonic_with_ntetra': corr > 0.85,
        'final_value': vals[-1] if vals else None,
        'max_value': max(vals) if vals else None,
        'slowdown_ratio': slowdown_ratio, # < 1 = ralentit, ~ 1 = lineaire,
        'saturating': slowdown_ratio is not None and slowdown_ratio < 0.5
    }

phi_analysis = analyze_observable('PHI', 'PHI')
psi_analysis = analyze_observable('PSI', 'PSI')
theta_analysis = analyze_observable('THETA', 'THETA')

result = {
    'seed': seed,
    'total_steps': total_steps,
    'time_s': total_time,
    'final_state': {
        'N': sim.N, 'E': sim.E,
        'n_tri': sim.n_tri, 'n_tetra': sim.n_tetra,
        'D_final': sim._compute_D(),
    },
    'first_edge_step': sim.first_edge_step,
    'first_triangle_step': sim.first_triangle_step,
    'first_tetrahedron_step': sim.first_tetrahedron_step,
    'coherent': coherent,
    'PHI_analysis': phi_analysis,
    'PSI_analysis': psi_analysis,
    'THETA_analysis': theta_analysis,
    'trajectory': trajectory,
}

save_path = OUTPUT_DIR + '/saut3_seed' + str(seed) + '.json'
try:
    with open(save_path, 'w') as f:
        json.dump(result, f)
except Exception as e:
    print(' WARN save : ' + str(e))

print(' Seed ' + str(seed) + ' DONE en ' + '{:.1f}'.format(total_time/60) +
print(' PHI : final=' + '{:.3f}'.format(phi_analysis['final_value'])
    + ' corr=' + '{:.3f}'.format(phi_analysis['corr_with_ntetra'])
    + ' slowdown=' + ('{:.3f}'.format(phi_analysis['slowdown_ratio'])) if
print(' PSI : final=' + '{:.3f}'.format(psi_analysis['final_value'])
    + ' corr=' + '{:.3f}'.format(psi_analysis['corr_with_ntetra'])
    + ' slowdown=' + ('{:.3f}'.format(psi_analysis['slowdown_ratio'])) if
print(' THETA : final=' + '{:.5f}'.format(theta_analysis['final_value'])

```

```

        + ' corr=' + '{:.3f}'.format(theta_analysis['corr_with_ntetra'])
        + ' slowdown=' + ('{:.3f}'.format(theta_analysis['slowdown_ratio'])) i

del sim
gc.collect()
return result

# =====
# ANALYSE GLOBALE 5 SEEDS
# =====
def analyze_saut3(results):
    print('')
    print('=' * 75)
    print(' ANALYSE GLOBALE SAUT 3 - CHALEUR BLOB REFORMULEE')
    print('=' * 75)

    n_seeds = len(results)

    def stats(vals):
        if not vals:
            return None
        m = float(np.mean(vals))
        s = float(np.std(vals))
        cv = (s/abs(m)*100) if m != 0 else 0.0
        return {'mean': m, 'std': s, 'cv': cv,
                'min': float(min(vals)), 'max': float(max(vals))}

    # Pour chaque candidat, extraire les indicateurs
    def evaluate_candidate(name, key):
        analyses = [r[key + '_analysis'] for r in results if r[key + '_analysis']]
        if not analyses:
            return None

        finals = [a['final_value'] for a in analyses if a['final_value'] is not N]
        corrs = [a['corr_with_ntetra'] for a in analyses if a['corr_with_ntetra']]
        slowdowns = [a['slowdown_ratio'] for a in analyses if a['slowdown_ratio']]

        n_monotonic = sum(1 for a in analyses if a['monotonic_with_ntetra'])
        n_saturating = sum(1 for a in analyses if a['saturating'])

        finals_stats = stats(finals)
        corrs_stats = stats(corrs)
        slowdown_stats = stats(slowdowns)

    # Tests :
    # T1 : monotonie 5/5 (corr > 0.85 sur 5 seeds)

```

```

T1_pass = (n_monotonic == n_seeds)
# T2 : reproductibilite finals CV < 30%
T2_pass = finals_stats and finals_stats['cv'] < 30
# T3 : saturation observee sur >= 4/5 seeds
T3_pass = (n_saturating >= n_seeds - 1)

# Score composite : 3 = parfait, 0 = aucun critere
score = sum([T1_pass, T2_pass, T3_pass])

return {
    'name': name,
    'finals_stats': finals_stats,
    'corrs_stats': corrs_stats,
    'slowdown_stats': slowdown_stats,
    'n_monotonic': n_monotonic,
    'n_saturating': n_saturating,
    'T1_monotonic_pass': T1_pass,
    'T2_reproducible_pass': T2_pass,
    'T3_saturating_pass': T3_pass,
    'score': score
}

phi_eval = evaluate_candidate('PHI', 'PHI')
psi_eval = evaluate_candidate('PSI', 'PSI')
theta_eval = evaluate_candidate('THETA', 'THETA')

print('')
print(' --- Tableau comparatif des 3 candidats ---')
print('')
print(' Critere | PHI | PSI | THETA')
print(' ' + '-' * 70)

def fmt_pass(b):
    return ' PASS ' if b else ' FAIL '

print(' T1 monotonie avec n_tetra (5/5) |' + fmt_pass(phi_eval['T1_monotonic']
+ '|' + fmt_pass(psi_eval['T1_monotonic_pass'])
+ '|' + fmt_pass(theta_eval['T1_monotonic_pass']))
print(' T2 reproductibilite (CV < 30%) |' + fmt_pass(phi_eval['T2_reproduci
+ '|' + fmt_pass(psi_eval['T2_reproducible_pass'])
+ '|' + fmt_pass(theta_eval['T2_reproducible_pass']))
print(' T3 saturation (>=4/5 seeds) |' + fmt_pass(phi_eval['T3_saturatin
+ '|' + fmt_pass(psi_eval['T3_saturating_pass'])
+ '|' + fmt_pass(theta_eval['T3_saturating_pass']))
print(' Score (sur 3) | ' + str(phi_eval['score'])
+ ' | ' + str(psi_eval['score'])
+ ' | ' + str(theta_eval['score']))

```

```

print('')
print(' --- Stats par candidat ---')
for cand in [phi_eval, psi_eval, theta_eval]:
    f = cand['finals_stats']
    c = cand['corrs_stats']
    sl = cand['slowdown_stats']
    print(' ' + cand['name'] + ' :')
    if f:
        print('      Final : mean=' + '{:.4f}'.format(f['mean'])
              + ' CV=' + '{:.1f}%'.format(f['cv']))
    if c:
        print('      Corr avec n_tetra : mean=' + '{:.4f}'.format(c['mean'])
              + ' min=' + '{:.4f}'.format(c['min']))
    if sl:
        print('      Ratio ralentissement : mean=' + '{:.3f}'.format(sl['mean'])

# Selection du gagnant
candidates = [phi_eval, psi_eval, theta_eval]
winner = max(candidates, key=lambda c: c['score'])

print('')
print('=' * 75)
print(' VERDICT SAUT 3')
print('=' * 75)

if winner['score'] == 3:
    print(' >> ' + winner['name'] + ' EST LA CHALEUR BLOB [M] verrouille')
    print(' >> Satisfait : monotonie + reproductibilite + saturation')
    print(' >> "D = chaleur" definitivement remplace par : ' + winner['name'])
elif winner['score'] == 2:
    print(' >> ' + winner['name'] + ' est la meilleure candidate (2/3 criter')
    print(' >> Validation [M] partielle')
else:
    print(' >> Aucune candidate ne satisfait pleinement les criteres')
    print(' >> Reformulation necessaire ou ouvre nouveau programme')

return {
    'PHI': phi_eval,
    'PSI': psi_eval,
    'THETA': theta_eval,
    'winner': winner['name'] if winner['score'] >= 2 else 'NONE',
    'winner_score': winner['score']
}

```

```
# =====
```

```

# MAIN
# =====
print('')
print('=' * 75)
print('  BLOB PRINCIPLE - SAUT 3 - CHALEUR BLOB REFORMULEE')
print('=' * 75)
print('  Question : trouver l observable qui remplace D comme chaleur')
print('  Candidats :')
print('    PHI - friction tetraedrique (paires de tetras voisins)')
print('    PSI - entropie locale moyenne (Shannon des degres voisins)')
print('    THETA - taux de transitions par tetraedre')
print('  Criteres :')
print('    T1 - Monotonie avec n_tetra (5/5)')
print('    T2 - Reproductibilite (CV < 30%)')
print('    T3 - Saturation (>= 4/5 seeds)')
print('=' * 75)

TOTAL_STEPS = int(os.environ.get('BLOB_TOTAL_STEPS', 5000000))
SEEDS_TO_RUN = [int(s) for s in os.environ.get('BLOB_SEEDS', '0,1,2,3,4').split('

print('')
print('  Configuration :')
print('    Pas par seed : ' + str(TOTAL_STEPS))
print('    Seeds : ' + str(SEEDS_TO_RUN))
print('    Duree estimee : ~' + str(10 * len(SEEDS_TO_RUN)) + ' min (PHI cher)')
print('=' * 75)

t_global = time.time()
all_results = []

for seed in SEEDS_TO_RUN:
    print('')
    print('-' * 75)
    print('  Seed ' + str(seed))
    print('-' * 75)
    try:
        r = run_seed_saut3(seed, TOTAL_STEPS)
        all_results.append(r)
    except Exception as e:
        print('  ERREUR : ' + str(e))
        import traceback
        traceback.print_exc()

global_time = time.time() - t_global

if len(all_results) >= 2:
    analysis = analyze_saut3(all_results)

```

```

report_path = OUTPUT_DIR + '/saut3_RAPPORT.json'
try:
    with open(report_path, 'w') as f:
        json.dump({
            'config': {
                'total_steps': TOTAL_STEPS,
                'seeds': SEEDS_TO_RUN,
                'global_time_s': global_time,
            },
            'analysis': analysis
        }, f, indent=2)
    print('')
    print(' Rapport final : ' + report_path)
except Exception as e:
    print(' WARN save report : ' + str(e))

print('')
print('=' * 75)
print('  CAMPAGNE SAUT 3 TERMINEE')
print('=' * 75)
print('  Temps : ' + '{:.1f}'.format(global_time/60) + ' min')
print('  Fichiers dans : ' + OUTPUT_DIR)
for f in sorted(os.listdir(OUTPUT_DIR)):
    sz = os.path.getsize(OUTPUT_DIR + '/' + f)
    print('      ' + f + ' (' + str(sz) + ' octets)')
print('=' * 75)

```